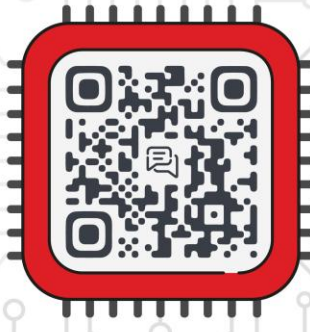




نظري

4



12



1440 sp

كلية الهندسة المعلوماتية

السنة الثالثة

Pipeline Hazards

د. سيرا أستور



RB Informatics; 3/11/2025

بنیان الحواسيب 2

مراجعة أفكار المحاضرة السابقة:

types of hazards

Structural hazard

Data hazard

تكلما في المحاضرة السابقة :

أن Pipeline هو أسلوب لتقسيم تنفيذ التعليمات إلى مراحل متعددة (stages) بحيث يمكن تنفيذ أكثر من تعليمة بنفس الوقت (بالتوازي).

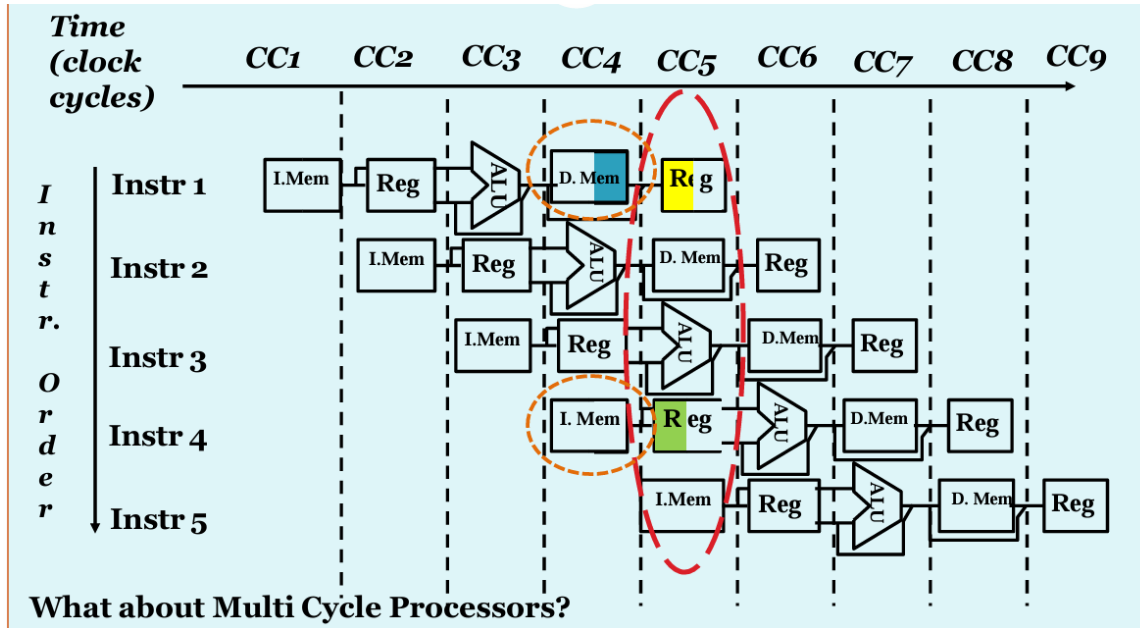
مثال على مراحل الـ Pipeline في معمارية MIPS:

- IF: Instruction Fetch
- ID: Instruction Decode
- EX: Execute
- MEM: Memory Access
- WB: Write Back

والمثال على هذه الفكرة ورد في المحاضرة السابقة (مثال الغسالة)

- ليس من الضروري أن تكون جميع مراحل الـ Pipeline متساوية في الزمن، وتحدثنا عن كيفية التراكب (overlapping) بين المراحل لتحسين الأداء.
- يتم أخذ أطول مرحلة (من حيث الزمن) وجعلها معياراً لتطبيق الزمن على جميع المراحل، مثل طريقة التنفيذ متعدد الدورات (Multicycle).
- في مرحلة ID يمكن تحقيق ربح في الأداء عندما تكون $n=4$, $n=5$ ، بحيث يكون $\text{Speedup} = 500\%$.
- عمق الـ Pipeline ويرمز له بـ $\text{Depth of Pipeline} = n$ (عدد المراحل).
- الـ Pipeline لا يكون دائماً مثالياً، لذلك نواجه بعض المشاكل (Hazards) ونحاول إيجاد حلول لها.
- الربح يكون أعظمي دائماً عندما تكون القيمة كبيرة.

Graphical Representation-SC:



- المخطط يوضح كيف تتداخل مراحل التعليمات المختلفة عبر الزمن حيث لدينا المحور الأفقي يمثل دورات الساعة (CC1, CC2, ...) والمحور العمودي يمثل ترتيب التعليمات (Instr1, Instr2, ...).
- نلاحظ أن كل تعليمة تبدأ مرحلة جديدة كل دورة ساعة، بحيث كل مرحلة تعمل بالتوازي مع مراحل من تعليمات أخرى فمثلاً:

المرحلة الأولى if:

جلب التعليمة من الذاكرة، ونزيد قيمة الـ PC حتى نجهز العنوان للتعليمة التالية، لذلك وضعنا بالـ $IM < cc1$

المرحلة الثانية id:

decode أي ن فك بنية التعليمة (opcode, rs, rt, rd...) ، ونقرأ القيم المطلوبة من الـ register file عندها نعلم ما تحتاجه التعليمة ALU ، Memory ، أو Register . من أجل ذلك وضعنا في الـ $IM, cc2$ Register.

ملاحظة:

الـ decode هو هاردوير لا يحتاج الى قطع، القطع المستخدمة فقط في الـ register file

المرحلة الثالثة EX:

الـ ALU تنفذ العملية المطلوبة (جمع، طرح، مقارنة...) وبحالة load أو store تحسب عنوان الذاكرة ومن أجل ذلك وضعنا في الـ $IM, cc3$ Register.

المرحلة الرابعة MEM:

إذا كانت التعليمة load نقرأ من الذاكرة وإذا كانت store نكتب على الذاكرة وإذا كانت التعليمة لا تحتاج الوصول للذاكرة نتخطى هذه المرحلة بدون عمل. ومن أجل ذلك وضعنا في الـ $cc4$ (M)

المرحلة الخامسة WB:

نكتب النتيجة للـ register المناسب سواء كانت النتيجة قادمة من ALU أو من Memory وتتم بالدورة cc5

والآن ماذا لو كان لدينا سيل من التعليمات ماذا سيحصل؟

في الـ cc4 يطلب القراءة مرتين من الـ memory وذلك بتعليمتين مختلفتين في نفس النبضة الزمنية وذلك سبب لدينا مشكلة (Hazard) وأيضا الـ memory complete في file register لأنه ليس جميع التعليمات تقوم بمرحلة الـ write back، (التعليمات التي لا تحتاج إلى wback هي التي تسبب الـ Hazard)

ملاحظة:

نلاحظ أنه في العنوان Graphical Representation-SC وهذا يعني أن الـ memory قطعة واحدة لا يتم فصلها عن بعضها

تعريف الـ Hazard :

أي مشكلة تسبب عدم مرور التعليمات التالية (can't complete instruction) وفي هذه المحاضرة سنتطرق إلى دراسة نوعين فقط والنوع الثالث في المحاضرة القادمة وهو الأعقد نوعا ما.

أولاً: Structural Hazard (الأشكال البنيوي):

يحدث عندما تريد أكثر من تعليمة استخدام نفس قطعة الهاردوير مثل الذاكرة أو الـ register file، (تعارض المصادر)

مثال:

في Single Cycle Processor إذا كانت هناك ذاكرة واحدة تُستخدم لكل من التعليمات والبيانات فإن تعليمة الـ load تحتاج قراءة من الذاكرة بينما تعليمة أخرى قد تحتاج جلب تعليمة جديدة من نفس الذاكرة وهذا يؤدي إلى تعارض (Hazard).

ومن الممكن ان يكون هناك أكثر من حل لهذه المشكلة :

- حل المشكلة عن طريق الهاردوير وهو حل سريع ولكن مكلف.
- الحل عن طريق الـ software ابطأ / اقل كلفة.

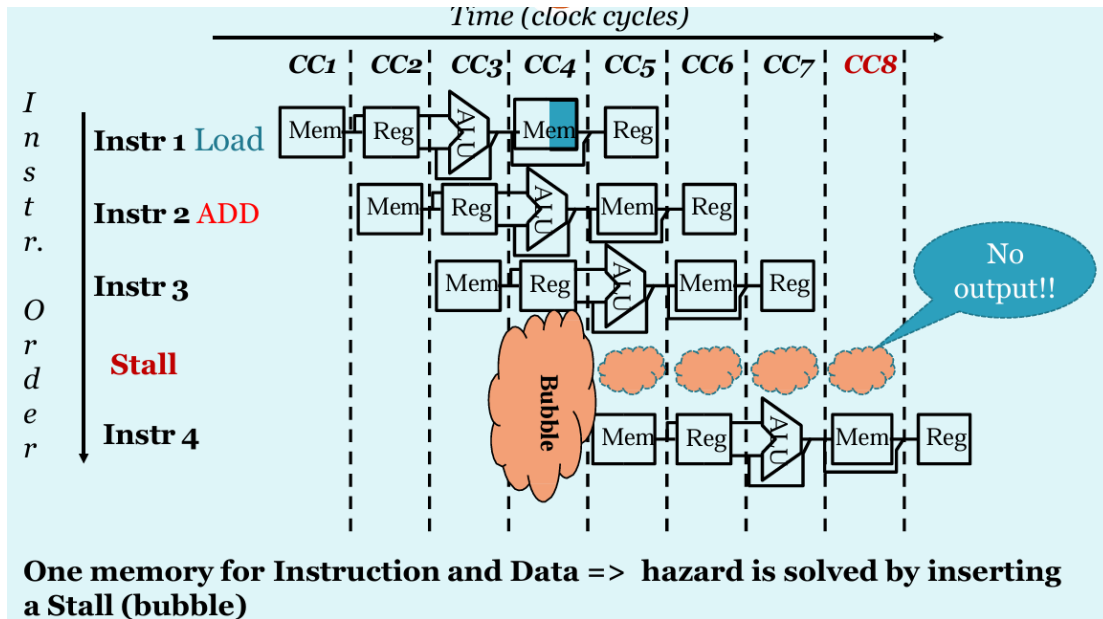
إيجاد الحل عن طريق الهاردوير:

في الـ multi cycle كان الحل هو تقسيم الذاكرة بحيث يكون قسم للـ data وقسم آخر للـ instruction، أما هنا في single cycle فسنضطر إلى إضافة قطع هاردوير إضافية.

الحل الثاني عن طريق software:

الحل الأول:

وهو وضع stall وهي تعليمة فارغة (nop) أي عبارة عن نبضات زمنية للتأخير بين التعليمات، ومن غير الضروري أن تكون الـ stall على نبضة زمنية واحدة. تسمى أيضاً (bubble)



- كما نلاحظ في التعليمية الأولى (inst1) في النبضة الرابعة (cc4) يتم الكتابة على الذاكرة، و بنفس النبضة تريد التعليمية الرابعة (inst4) الجلب من الذاكرة.
- لذلك يحصل stall لحين الكتابة على الذاكرة، وتبدأ التعليمية الرابعة في النبضة الخامسة (cc5) بدل الرابعة
- عدم تمرير قيمة بالدخل يؤدي إلى عدم وجود الخرج فمثلا في الـ cc4 لم يتم تمرير قيمة لذلك في الـ cc8 لا يوجد خرج.
- يكون الخرج جاهز عندما تدخل التعليمية مرحلة WB أي cc9 حسب المخطط.
- حسب المخطط الـ Bubble نلاحظ أن :
 - Cc5 هو خرج الـ cc1, Cc6 هو خرج الـ cc2
 - Cc7 هو خرج الـ cc3
 - في cc8 لا يوجد خرج لأننا أضفنا الـ Bubble. الخرج سيكون بالـ Cc9

Stall degrades the performance

مثال:

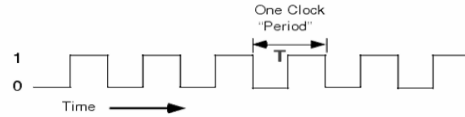
لدينا نسبة التعليمات التي تحتاج الوصول للبيانات (loads/stores): $f=40\%=0.4$ ولدينا كل تعليمة تحتاج دورة واحدة إضافية عند حدوث الـ stall إذن $p=1$ دورة ولدينا الـ cpi لكل تعليمة هو 1 (single cycle) وسرعة ساعة المعالج (معدل النبضة) الذي فيه الـ structural hazard أعلى بنسبة 1.05 من المعالج العادي. ما هو وقت تنفيذ كل تعليمة؟

$$\text{CPUtime} = \text{CC} * T = \text{CC} / f$$

$$\text{CPI} = \text{CC} / \text{IC} \Rightarrow \text{CC} = \text{IC} * \text{CPI} \Rightarrow$$

$$\text{CPUtime} = \text{IC} * \text{CPI} / f$$

(Frequency) f تردد المعالج
 $1/T =$
 IC عدد التعليمات في البرنامج
 CC عدد الأدوار الكلية في البرنامج
 CPI عدد الدورات (الوسطي) في التعليم
 (Cycle Per Instruction)



$$\text{Speedup} = \frac{\text{Performance}(\text{new})}{\text{Performance}(\text{old})} = \frac{\text{CPUtime}(\text{old})}{\text{CPUtime}(\text{new})}$$

Amdahl's Law

1. نحسب ال CPI للمعالج مع ال hazard:

$$\text{CPI}_{\text{hazard}} = 1 + f * p = 1 + 0.4 \times 1 = 1.4$$

2. زمن دورة المعالج ذو ال hazard:

$$\text{ClockCycleTime}_{\text{hazard}} = \text{ClockCycleTime}_{\text{ideal}} / 1.05$$

3. زمن التعليم المتوسط لكل معالج:

المعالج المثالي :

$$\text{ClockCycleTime}_{\text{ideal}} * 1 = T$$

المعالج ذو hazard:

$$T_{\text{hazard}} = 1.4 \times \text{ClockCycleTime}_{\text{ideal}} / 1.05$$

نسبة زمن التعليم (hazard) مقابل (ideal) :

$$T_{\text{hazard}} / T_{\text{ideal}} = 1.4 / 1.05 = 1.3333$$

أي أن زمن تعليم المعالج الذي فيه ال hazard أطول من زمن التعليم في المعالج المثالي أي حوالي 33.33% أطول. لذلك، المعالج بدون ال structural hazard أسرع من المعالج الذي يعاني منه بنسبة 1,333.

الحل الثاني (Adding Functional Units):

بعض المعالجات تحصل فيها مشكلة Structural Hazard لأن وحدة واحدة فقط تخدم عمليتين مختلفتين في وقت واحد (مثل القراءة من الذاكرة والكتابة إليها) وإذا أضفنا وحدات إضافية (مثل استخدام Dual-port memory بدل Single-port memory يمكن تنفيذ أكثر من عملية ذاكرة في نفس الدورة، وبالتالي نتفادى ال stalls. سنتابع بناء على معطيات المثال السابق بحيث لدينا أيضا :

Machine A	تستخدم ذاكرة ثنائية المنفذ Dual-ported memory (أي لا يوجد Hazard)
Machine B	تستخدم ذاكرة أحادية المنفذ Single-ported memory (يوجد Hazard و B أسرع بـ 1.05 مرة من A)
Ideal CPI = 1	لكل من A و B
Loads/Stores = 40%	أي أن 40% من التعليمات تتعلق بالذاكرة، وهي التي تسبب التعارض في Machine B

الحل:

بما أن A لا تعاني من Hazard فسرعتها تعتمد على عمق الـ pipeline فقط:

$$SpeedUpA = \frac{Pipeline\ Depth}{(1 + 0)} = Pipeline\ Depth$$

و B لديها مشاكل Hazard تؤثر على 40% من التعليمات:

$$CPI = 1 + 0.4 \times 1 = 1.41$$

ولأن ساعة Machine B أسرع بـ 1.05 فزمن الدورة يقل بـ 1/1.05

$$SpeedUpB = \frac{Pipeline\ Depth}{1.4} * 1.05 = Pipeline\ Depth * 0.75$$

$$SpeedUpA/SpeedUpB = \frac{Pipeline\ Depth}{0.75 \times Pipeline\ Depth} = 1.33$$

ثانياً: Data Hazard:

في pipeline كل تعليمة (instruction) تنفذ على المراحل التي ذكرناها سابقاً والـ Data Hazard يحصل عندما تعليمة جديدة تريد استخدام بيانات (data) لم تنتهي التعليمة القديمة من إنتاجها (كتابتها)

● Example: consider two instructions:

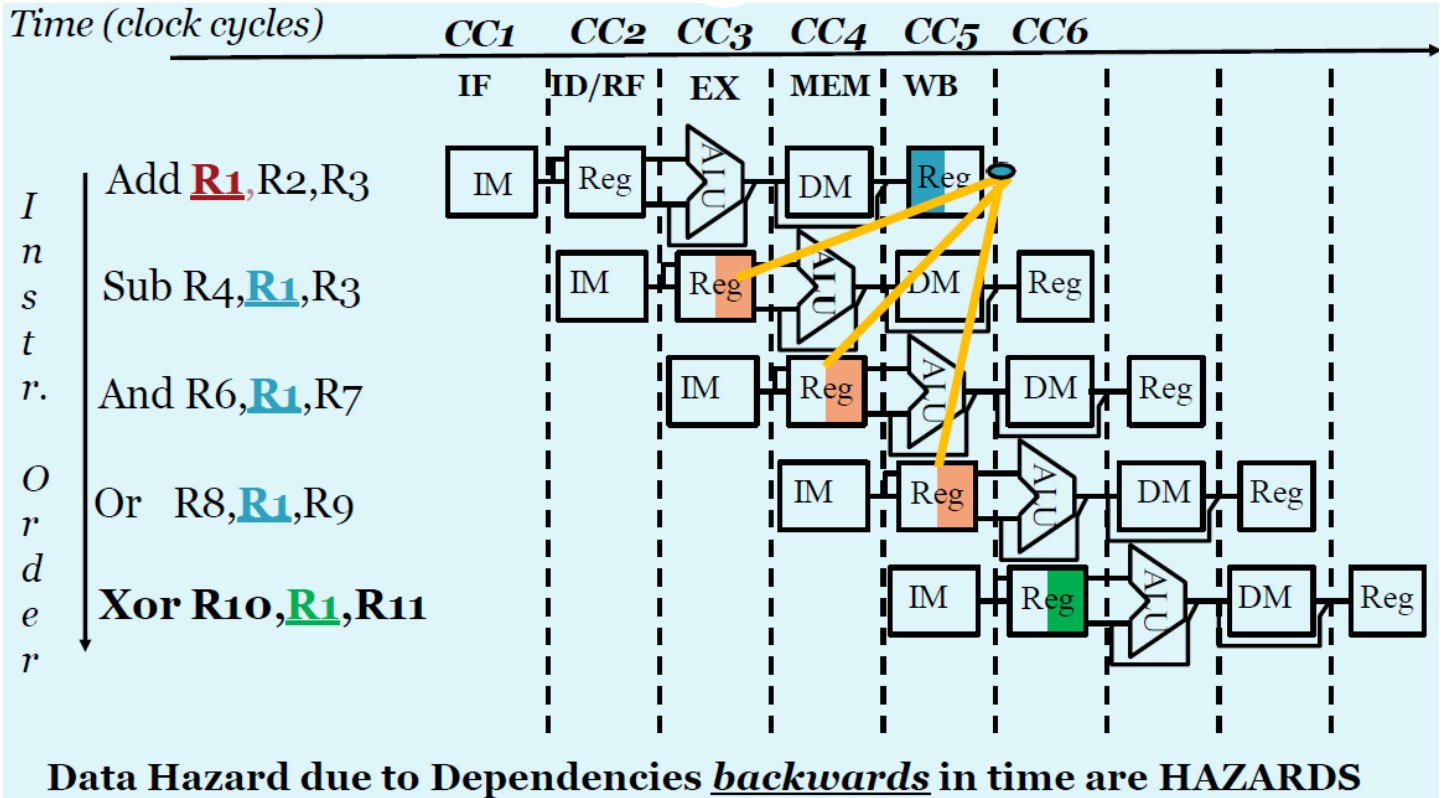
```
add    $s0, $t0, $t1
sub     $t2, $s0, $t3    # needs $s0
```

فمثلاً التعليمة الثانية (sub) بحاجة لقيمة \$s0، ولكن الـ add لم تنتهي من كتابة النتيجة لذلك حصل Data Hazard مثال آخر:

```
1: Add  R1, R2, R3
2: Sub  R4, R1, R3
3: And  R6, R1, R7
4: Or   R8, R1, R9
5: Xor  R10, R1, R11
```

لدينا المخطط التالي فيه التعليمات السابقة وكل كل تعليمة تمرّ بالمراحل الخمسة (وكل صف هو تعليمة)

IF → ID → EX → MEM → WB



▪ التعليم الأولى ADD تنتج R1

▪ كل التعليمات بعدها (SUB, AND, OR, XOR) تريد استخدام R1 قبل الانتهاء من كتابتها لذلك أصبح لدينا Data Hazard بين كل تعليمة والأخرى.

إذن ماذا يحصل زمنياً بالتفصيل؟

▪ في Cycle الأولى: الـ ADD تنفذ مرحلة IF وفي Cycle الثانية ADD: تنفذ ID و SUB تنفذ IF

وهكذا التعليمات تستمر بالتنفيذ ولكن أين يحصل الـ Hazard فعلياً؟

▪ السهم البرتقالي: يرمز لاعتماد الـ SUB على ناتج الـ ADD ولكن ADD من كتابته (R1 سوف يكتب بالـ

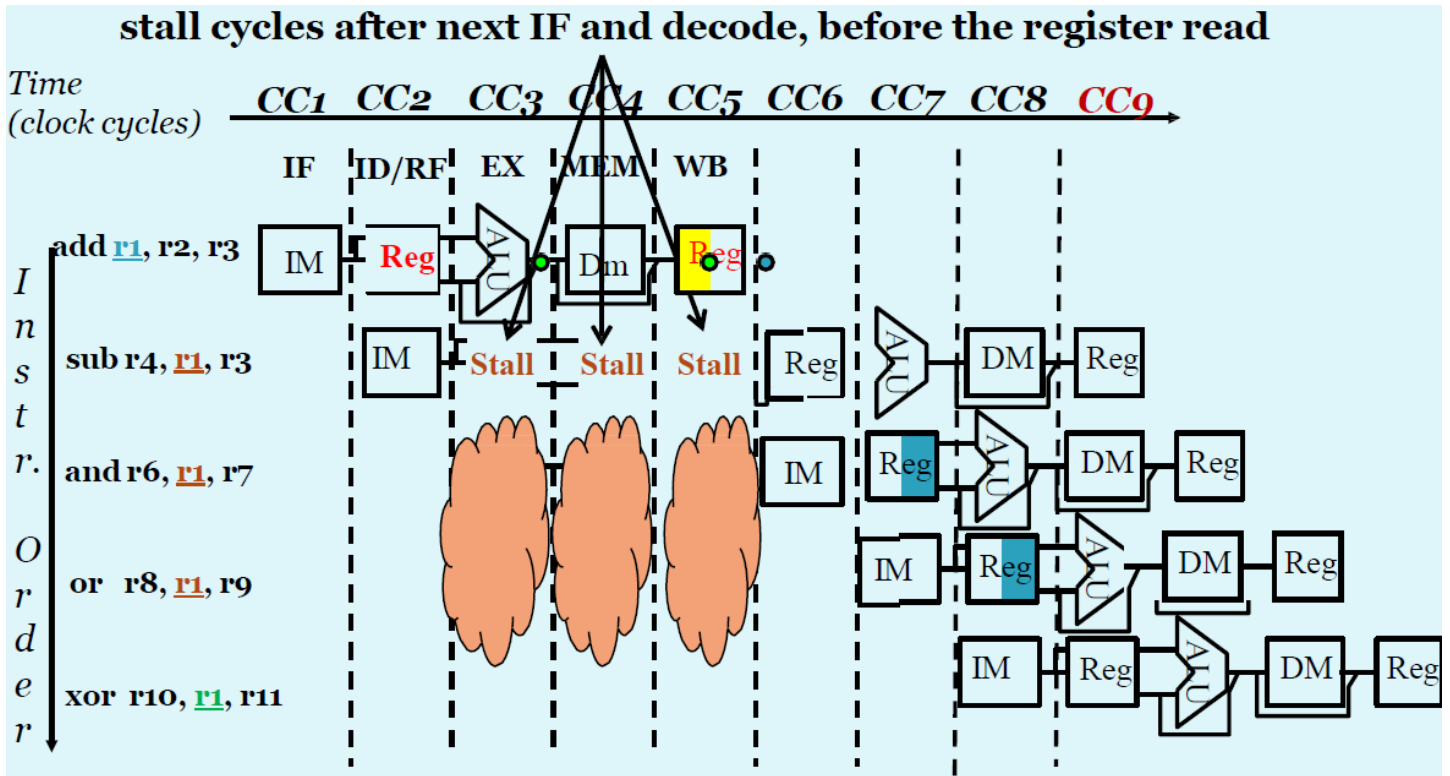
WB بعد كم دورة في cc5) وكذلك السهم البرتقالي من ADD إلى AND يحدث أيضاً Hazard لأن AND

تحتاج R1 ونفس الشيء من ADD إلى OR

ملاحظة:

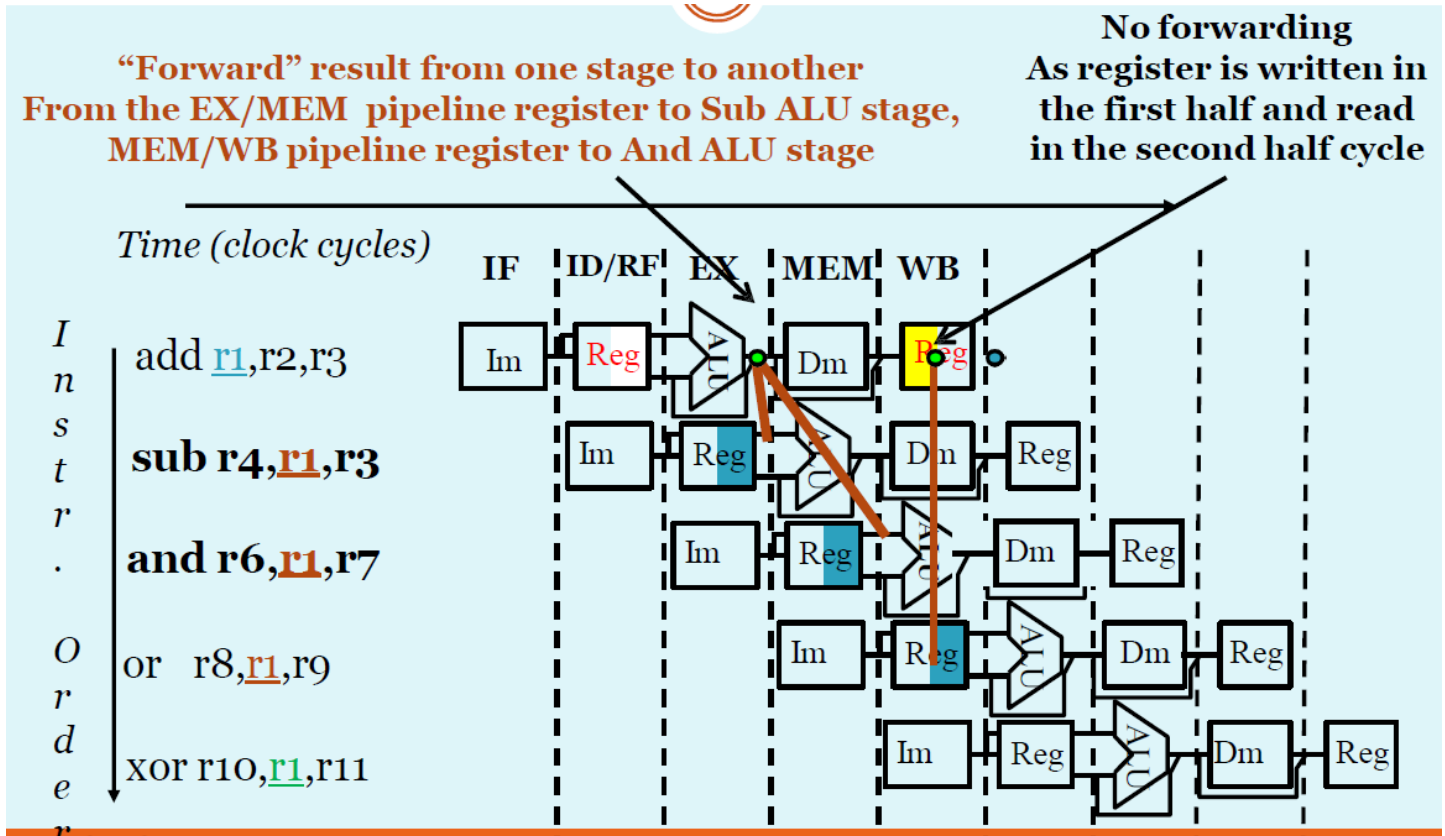
عندما يتوجه سهم للخلف أي يوجد اعتماد زمني للخلف وهذا يعني وجود Hazard

👉 كيف نحل هذا الـ Hazard؟



- نلاحظ بعد مرحلة الـ ID للتعليمية الثانية (SUB) يوجد دوائر فارغة (Bubbles)
- هذه الدوائر معناها Stall أو انتظار أي أن SUB دخلت مرحلة الـ ID ولكن لم تصل لمرحلة EX لأن النتيجة من ADD (R1) لم تنتهي بعد، فالمعالج يختار إيقافها (stall) ريثما ADD تكتب النتيجة بـ WB.
- لكن هذه المشكلة Stalls تقلل الأداء لأنه هناك إضاعة دورات (Cycles) بدون تنفيذ فعلي.

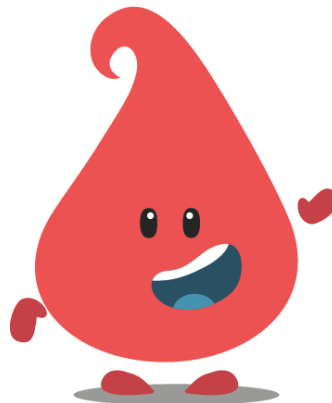
الحل الثاني Forwarding:



- بدلاً من انتظار النتيجة و مرحلة WB للتعليلة الأولى نستطيع أخذها مباشرة من مرحلة EX/MEM أو MEM/WB لمرحلة EX في التعليلة التالية باستخدام ال Forwarding
- نلاحظ الأسهم الحمراء التي تمثل خطوط Forwarding أي أن النتيجة تنتقل مباشرة من ALU التي هي نتيجة ال ADD لتدخل على ALU التابعة لتعليلة SUB
- أي بالمرحلة التالية ADD قامت بحساب الناتج بـ EX , و SUB دخلت EX بعد دورة، وتحتاج نفس الناتج. فبدلاً من انتظارها حتى تكتب بال Register File . المنفذ ALU output يذهب مباشرة عبر سلك Forward إلى دخل مرحلة ALU الخاصة بالتعليلة الثانية و هكذا SUB تعمل فوراً بدون Stall

ملاحظة:

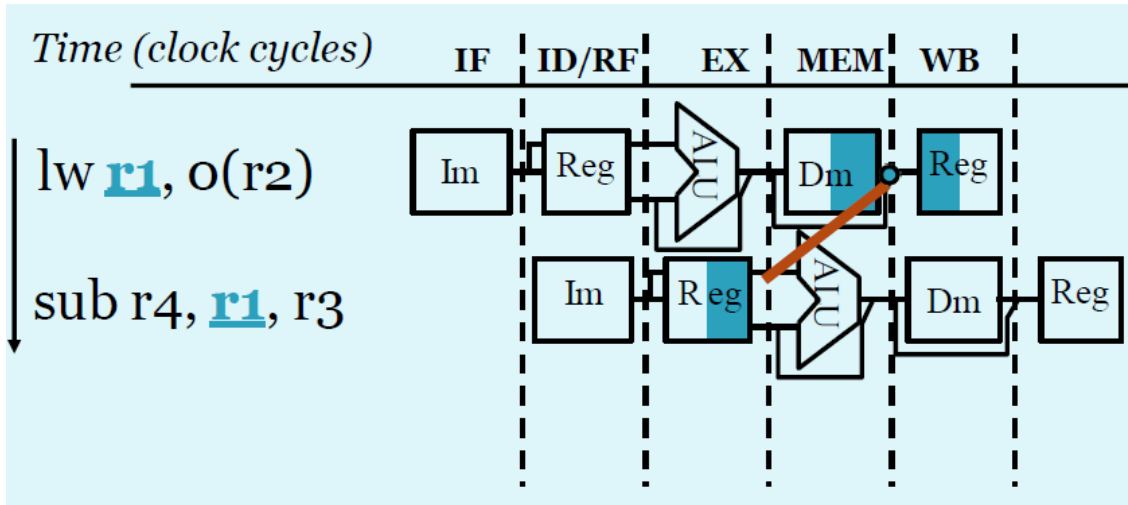
Forwarding حل المشكلة بدون تأخير، ولكن لا يستطيع إيجاد حل لكل الحالات مثل ال Load



Data Hazard Load-Us

وهي حالة خاصة عندما تستخدم التعليمة التالية النتيجة التي تأتي من Load هنا سيلزم إضافة Stall حتى مع الـ Forwarding

فمثلاً بالمخطط لدينا :



الـ sub تحتاج قيمة \$s0 لكن الـ lw لا تأتي بالبيانات إلا بمرحلة MEM وبالتالي فإن البيانات ليست جاهزة في مرحلة EX . الحل الوحيد هو إضافة (Stalls) بينهما

نلاحظ أن الـ lw تنفذ عبر المراحل (IF-ID-EX-MEM-WB)

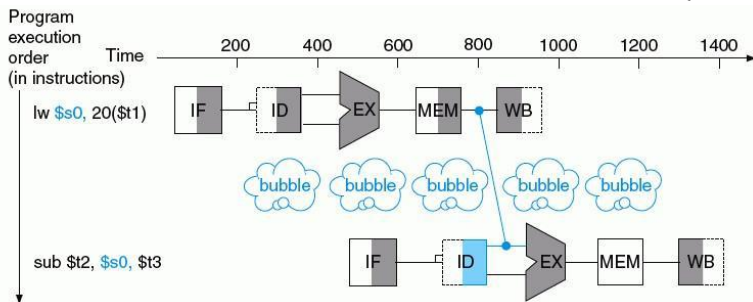
والـ sub بعدها مباشرة

نضيف بينهما فقاعات (Bubbles) بعد

مرحلة الـ ID. هذه الـ Stalls مهمة لأنه

من غير الممكن أن نقوم بـ Forward من

الذاكرة قبل أن نقوم بتحميل البيانات :



Forwarding Unit Hardware

أي نريد معرفة كيف المعالج من الداخل يعلم انه يوجد Hazard وكيف يقرر عمل Forwarding يعني يمرر النتيجة من مرحلة متقدمة لتعليمات لاحقة بدون انتظار الكتابة في الـ Register File

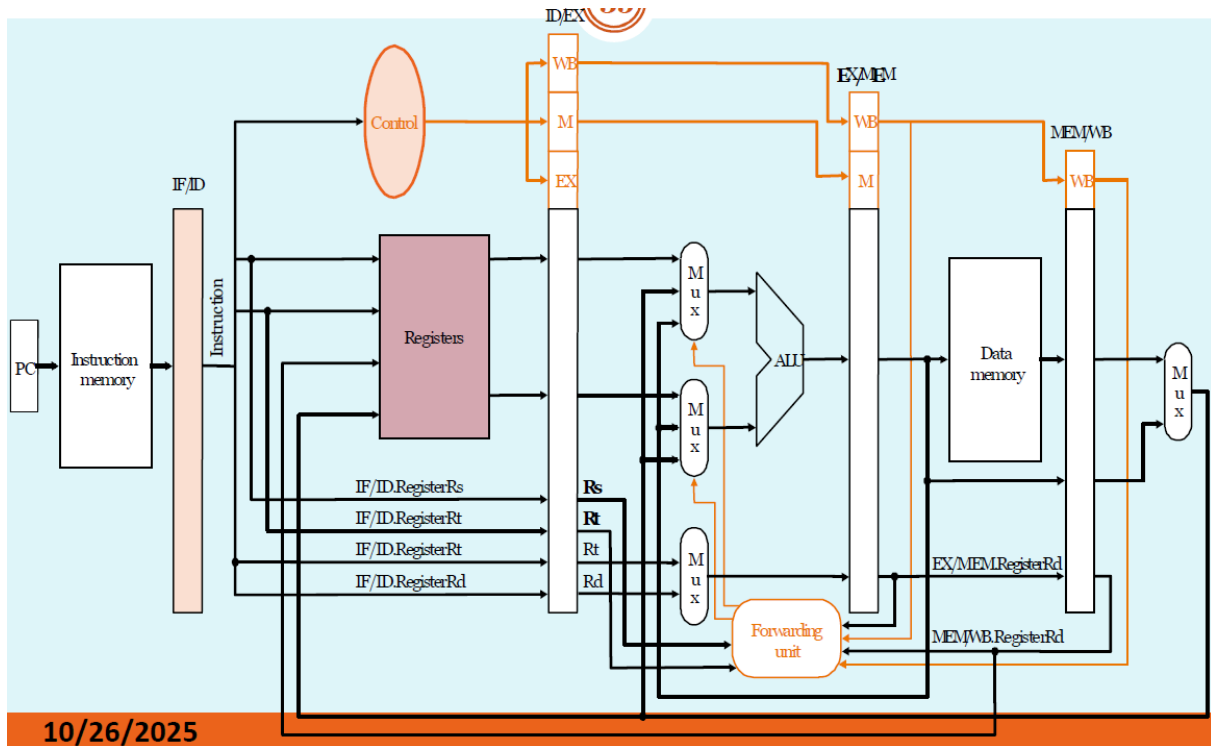
الـ Forwarding Unit تعمل تحديداً على المراحل الثلاثة الأخيرة:

EX مرحلة التنفيذ (ALU)

MEM مرحلة الوصول للذاكرة

WB مرحلة كتابة النتيجة للـ Register File

لماذا؟ لأن القيم الناتجة بتطلع من EX أو MEM و التعليمات الجديدة بالـ EX تحتاج نفس القيم فوراً



■ في المخطط نجد:

على اليمين المراحل WB, MEM, EX كل مرحلة فيها سجلات مرحلية مثل:

■ EX/MEM pipeline register

■ MEM/WB pipeline register

هذه السجلات تخزن القيم المؤقتة بين كل مرحلتين

وفي الوسط المربع الكبير (ALU) وهذه الوحدة هي التي تنفذ العمليات الحسابية والمنطقية (add, sub, and, or.....)

■ ولها مدخلين (Input A, Input B)

■ الناتج (ALU Result) يذهب منها نحو اليمين (EX/MEM register)

المربع الصغير الموجود أسفل ALU وهو Forwarding Unit وفيه إشارات ForwardA / ForwardB

وظيفتها:

■ عندها مداخل من سجلات المراحل السابقة EX/MEM و MEM/WB

■ تقارن هل الـ Register الذي خرج من التعليمة السابقة هو نفسه الذي التعليمة الحالية تريد استخدامه كمدخل A أو B؟

■ إذا نعم تفعل إشارة ForwardA = 1 أو ForwardB = 1

هذه الإشارات تتحكم بـ Multiplexers (MUXes) قبل الـ ALU

Multiplexers

قبل مدخل ALU توجد MUXes وكل MUX يأخذ 3 احتمالات للمصدر:

1. القيمة العادية من Register File إذا ما في hazard

2. القيمة من EX/MEM إذا النتيجة أتت من التعليمة قبلها مباشرة

3. القيمة من MEM/WB إذا النتيجة اتت من تعليمة أبعد
وإشارة التحكم ForwardA أو ForwardB تختار أي مدخل تستخدم

مثلا لدينا :

add R1, R2, R3 # تنتج R1

sub R4, R1, R5 # تحتاج R1

الدورة الأولى:

ADD في EX تحسب الناتج لـ R1 و SUB في ID بعدها مباشرة

الدورة الثانية:

- ADD تكمل لـ MEM والنتيجة موجودة في سجل EX/MEM
- SUB تدخل EX وتحتاج Forwarding Unit $R1 \rightarrow R1$ تكشف التطابق R1 نفسه تفعل $ForwardA = 10$ يعني خذ القيمة من (EX/MEM)
- ALU لتعليمة SUB تستعمل النتيجة مباشرة عبر الخط الأحمر وبذلك نكون :
- SUB نفذت بدون أي Stall / لم ننتظر كتابة R1 في WB

Discovering Forwarding

كيف المعالج يقوم باكتشاف متى يحتاج البرنامج لـ Forwarding؟

- يجب أن يكتشف إذا كانت التعليمة الجديدة تحتاج لاستخدام ناتج من تعليمة سابقة لم يكتب بالـ Register File بعد.
 - المعالج لديه وحدة اسمها Forwarding Unit وهي التي تقرر إذا سيتم عمل forwarding أو لا.
 - تقارن destination register الخاص بالتعليمة القديمة مع source registers الخاص بالتعليمة الجديدة، فإذا كلن نفس الـ register ، أي يوجد hazard ويجب القيام ب forwarding
- $$if (EX/MEM.RegWrite) \wedge (EX/MEM.RegisterRd \neq 0) \wedge (EX/MEM.RegisterRd = ID/EX.RegisterRs) then ForwardA = 10$$

تعني:

- لو أردنا في المرحلة السابقة (EX/MEM) أن نكتب على سجل $RegWrite = 1$ (تم تفعيله)
- هو نفس السجل الذي التعليمة الحالية تريد القراءة منه كـ Rs
- إذاً، فعل $ForwardA = 10$ ، أي خذ القيمة من (EX/MEM)

```

if (MEM/WB.RegWrite) ^
(MEM/WB.RegisterRd ≠ 0) ^
(MEM/WB.RegisterRd = ID/EX.RegisterRt)
then ForwardB = 10

```

نفس الفكرة لكن من مرحلة أبعد (MEM/WB)

Software solution

compiler يضيف NOPs (تعليمات انتظار) ليمنع التضارب ومن الممكن وضع NOPs في حال وجود الـ forwarding ولكن الـ Hazard لم يعالج نهائياً

مثال:

نلاحظ أن كل التعليمات بعد sub مباشرة تستخدم \$2 الناتجة من التعليمات السابقة وهذا يعني Data Hazards متتالية

```

sub $2, $1, $3
and $12, $2, $3
or $13, $6, $2
add $14, $2, $2
sw $13, 100($2)

```



كل مرة بعد تعليمة تنتج قيمة، نضع NOP لماذا؟ حتى يعطي وقت كافي للمعالج يكتب النتيجة قبل ما تحتاجها التعليمة التالية.

```

sub $2, $1, $3
nop
and $12, $2, $3
nop
or $13, $6, $2
nop
add $14, $2, $2
nop
sw $13, 100($2)

```

Avoiding Stall by Code Reorder

الفكرة:

بدلاً من إيقاف التنفيذ ممكن نرتب الكود بحيث التعليمة لا تعتمد مباشرة على خرج التعليمة السابقة.

مثال:

لدينا الكود المكتوب بلغة C++:

```

A = B + E;
C = B + F;

```



```

lw $t1, 0($t0)
lw $t2, 4($t0)
add $t3, $t1, $t2 # A = B + E ← hazard محتمل
lw $t4, 8($t0)
add $t5, $t1, $t4 # C = B + F
sw $t3, 12($t0)
sw $t5, 16($t0)

```

نلاحظ أن `add $t3` بعدها فوراً تعليمة `lw $t4` التي لا تعتمد على `$t3` لكن بعدها `add $t5` تعتمد على نفس `$t1` الذي كان مستخدم بالـ `add` الأولى، إذا يوجد لدينا hazard الحل باستخدام Code Reorder :

```
lw $t1, 0($t0)
lw $t2, 4($t0)
lw $t4, 8($t0) # قبل وضعها
add $t3, $t1, $t2
add $t5, $t1, $t4
sw $t3, 12($t0)
sw $t5, 16($t0)
```

بعد الترتيب أصبح هناك تعليمات مستقلة بين العمليات التابعة لنفس السجل. أي أن المعالج أصبح لديه الوقت الكافي لينفذ ويكتب النتائج بدون Stall

ملاحظة :

- وجود تعليمتين `sw` متتاليتين لا يعطي Hazard
- في الـ mips لدينا 2 forward نضعهم على سجلين مختلفين هما `rs,rt`



THE END